

作业 5 fork 系统调用

同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 毛星博 学号 2351520

一、接作业 2



T0 时刻，用户区如图所示（注释：阴影部分已分配，除了 PA 正文段和可交换部分，还紧贴存放有其它进程的正文段和可交换部分）。PA 进程代码段 3 页，数据段 1 页，堆栈段 1 页；x_caddr=0x402000，p_addr=0x407000。PA 创建子进程 PB。

1、以此情景为背景，手撸、整理、注释 fork 系统调用源代码。

1.1 系统调用入口函数 Sys_Fork()

```
int SystemCall::Sys_Fork()
{
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
    procMgr.Fork(); // 调用进程管理器的 Fork 函数

    return 0;
}
```

1.2 Fork 主函数 ProcessManager::Fork()

```
void ProcessManager::Fork()
// Fork 系统调用的主入口函数，负责寻找空闲 PCB，调用 NewProc 创建子进程
{
    User& u = Kernel::Instance().GetUser();
    Process* child = NULL;;

    /* 寻找空闲的 process 项，作为子进程的进程控制块 */
    for ( int i = 0; i < ProcessManager::NPROC; i++ )
    {
        if ( this->process[i].p_stat == Process::SNULL )
        {
            child = &this->process[i]; // 找到空闲 PCB，记录其地址

            break;
        }
    }
}
```

```

}
// 如果没有找到空闲 PCB
if (child == NULL)
{
    u.u_error = User::EAGAIN; // 设置错误码为 EAGAIN
    return; // 直接返回，fork 失败
}

if ( this->NewProc() ) /* 子进程返回 1，父进程返回 0 */
{
    /* 子进程 fork()系统调用返回 0 */
    u.u_ar0[User::EAX] = 0; //设置子进程的 fork()返回值为 0
    u.u_cstime = 0; //子进程在核心态消耗的累计时间
    u.u_stime = 0; //本进程在核心态消耗的时间
    u.u_cutime = 0; //子进程在用户态消耗的累计时间
    u.u_utime = 0; //本进程在用户态消耗的时间
    // 子进程是新创建的，所以这些时间都应该从 0 开始
}
else
{
    /* 父进程进程 fork()系统调用返回子进程 PID */
    u.u_ar0[User::EAX] = child->p_pid;
}
return;
}

```

1.3 创建子进程核心 - ProcessManager::NewProc()

```

int ProcessManager::NewProc()
{
    Process* child = 0;
    // 遍历进程表，寻找空闲 PCB
    for (int i = 0; i < ProcessManager::NPROC; i++)
    {
        if (process[i].p_stat == Process::SNULL)
        {
            child = &process[i]; // 找到空闲 PCB，记录其地址
            break;
        }
    }
    if (!child)
    {
        Utility::Panic("No Proc Entry!"); // 系统崩溃，打印错误信息
    }
}

```

```

// 获取当前进程（父进程）的 u 区和进程指针
User& u = Kernel::Instance().GetUser();
Process* current = (Process*)u.u_procp; // current 指向父进程 PA

// 调用 Clone(), 将父进程的 PCB 数据复制给子进程
// Clone 会复制: p_size, p_uid, p_nice, p_textp 等
// 会增加: 文件引用计数、正文段引用计数、工作目录引用计数
current->Clone(*child);

// 保存父进程的现场到 u.u_rsav
// SaveU(u.u_rsav)保存 ebp 和 esp 到 u.u_rsav 中
// 父进程返回时, 从这里的下一条指令继续执行;子进程首次被调度时, 也会从这里的
下一条指令开始执行
SaveU(u.u_rsav);

// 备份父进程的页表指针, 因为接下来要为子进程分配新的页表
PageTable* pgTable = u.u_MemoryDescriptor.m_UserPageTableArray;

// 为子进程分配新的页表空间 (8KB, 2 张页表)
u.u_MemoryDescriptor.Initialize();

// 复制父进程的相对地址映照表给子进程。PA 的代码段(3 页)、数据段(1 页)、栈段(1
页)的页表项都被复制。意味着 PB 初始时拥有和 PA 一模一样的虚存布局。
if (NULL != pgTable)
{
    // MemCopy 参数: (源地址, 目标地址, 大小)
    Utility::MemCopy((unsigned long)pgTable,
                     (unsigned long)u.u_MemoryDescriptor.m_UserPageTableArray,
                     sizeof(PageTable) *
MemoryDescriptor::USER_SPACE_PAGE_TABLE_CNT);
}

// 临时将 u.u_procp 指向子进程, 这样复制后, 子进程的 u 区就有了正确的进程指针
u.u_procp = child;

// 获取用户页管理器 (负责分配用户态内存)
UserPageManager& userPageManager = Kernel::Instance().GetUserPageManager();

// 准备复制的源地址和目标地址
unsigned long srcAddress = current->p_addr; // 父进程图像起始地址 = 0x407000
// 为子进程分配内存
// current->p_size 是进程图像大小 (单位: 4KB 页)
// 包括 PPDA + 数据段 + 堆栈段, 共 3 页 12KB
unsigned long desAddress = userPageManager.AllocMemory(current->p_size);

```

```

// 检查内存分配是否成功
if (desAddress == 0) /* 内存不够，需要交换到磁盘 */
{
    // 设置父进程状态为 SIDL（创建中）
    current->p_stat = Process::SIDL;

    // 子进程暂时指向父进程图像地址
    child->p_addr = current->p_addr;

    // 保存现场到 u.u_ssav（换入时的返回点）
    // 当子进程从交换区换入时，会从这里继续执行
    SaveU(u.u_ssav);

    // 调用 XSwap 将子进程换出到交换区
    this->XSwap(child, false, 0);

    // 设置 SSWAP 标志，表示子进程需要从交换区换入
    child->p_flag |= Process::SSWAP;

    // 恢复父进程状态为 SRUN（就绪）
    current->p_stat = Process::SRUN;
}
else
{
    /* 内存充足，直接在内存中复制 */
    // 获取进程图像大小（页数）
    int n = current->p_size;

    // 设置子进程的图像起始地址
    child->p_addr = desAddress;

    // 逐页复制父进程图像到子进程
    while (n--)
    {
        Utility::CopySeg(srcAddress++, desAddress++);
    }
}

// 恢复 u.u_procp 为父进程
u.u_procp = current;

// 恢复父进程的页表指针
u.u_MemoryDescriptor.m_UserPageTableArray = pgTable;

```

```

// 返回 0 给父进程
// 子进程的返回值是 1，来自于 Swtch()函数的返回值
// 当子进程首次被调度时：
// 1. Swtch()恢复子进程的 u.u_rsav
// 2. Swtch()返回 1
// 3. 子进程从 SaveU 的下一句开始执行
// 4. return 的值是 Swtch()的返回值 1，而不是这里写的 0
return 0;
}

```

内存布局：

父进程 PA:

正文段: 0x402000 - 0x404FFF (3 页, 12KB, 共享的, 由 x_caddr 指向)

可交换部分: 0x407000 - 0x409FFF (3 页, 12KB, 由 p_addr 指向)

子进程 PB (假设分配到 0x40A000) :

正文段: 0x402000 - 0x404FFF (共享父进程的, 不需要复制)

可交换部分: 0x40A000 - 0x40CFFF (3 页, 12KB, 独立复制)

2、子进程图像创建成功后，描述父进程 Fork 系统调用返回过程。

- ① NewProc() 执行完毕返回 0
- ② 回到 Fork() 函数：因为父进程中 NewProc() 返回值为 0，所以进入 else 分支
- ③ 设置返回值：将 u.u_ar0[User::EAX] 设置为子进程 PB 的 PID。这是为了让用户态的 pid = fork() 拿到子进程 ID。
- ④ 函数返回：Fork() 函数执行完毕，通过 Sys_Fork 返回到系统调用处理函数 Trap1() / Trap()。
- ⑤ 优先级重置：在 Trap 返回用户态前，会检查 u.u_error (错误码)、执行 Setpri()。因为执行系统调用（下半部）时优先级可能被动态调整（如 I/O 等待），返回前需要恢复应用程序的优先级（大于 100 的值）。
- ⑥ 返回到 SystemCallEntrance()，检查 RunRun > 0 (是否需要进程切换)。如果需要，调用 Swtch(); 否则继续
- ⑦ 恢复现场：RestoreContext()，执行 IRET 指令，恢复用户态寄存器（CS, EIP, ESP, SS 等）。回到用户态
- ⑧ PA 进程继续执行 int 0x80 之后的指令，此时 EAX 寄存器中已经是子进程的 PID。

3、描述子进程第 1 次被选中，投入运行的过程。

- ① 某个时刻触发进程切换，Swtch()被调用
- ② 保存当前进程现场: SaveU(u.u_rsav)
- ③ 切换到 0# 进程: SwtchUStruct(procZero); RetU();
- ④ Select()选择最高优先级进程 -> 选中子进程 PB (子进程 p_pri=0, 优先级最高)
- ⑤ 切换到子进程: SwtchUStruct(selected); RetU();
- ⑥ 恢复子进程的 u 区和页表:

u = 子进程的 u 区

MapToPageTable() - 建立子进程的地址映射

⑦ 检查 p_flag & SSWAP

如果子进程被换出过，aRetU(u.u_ssav) 从换入点返回

如果在内存中，不执行此分支

⑧ Swtch() return 1

子进程第一次运行时，在 Swtch() 的汇编代码中，会显式执行 `mov 1, %eax`。这使得 PB 在恢复执行时，寄存器 EAX 的值为 1，Swtch() 返回 1

⑨ 返回到 NewProc() 中 SaveU(u.u_rsav) 的下一条指令

(因为子进程的 u.u_rsav 是从父进程复制的)

⑩ NewProc() 继续执行后续代码，最终 return，返回 Fork() 函数

⑪ Fork() 中: if (this->NewProc()) 条件为真 (因为 Swtch 返回 1)

执行 `u.u_ar0[User::EAX] = 0`

⑫ Fork() return -> Sys_Fork() return

⑬ SystemCall::Trap() -> RestoreContext() -> iret

⑭ 子进程返回用户态，fork() 返回值为 0

从 fork() 调用的下一条指令继续执行

4 (选做) 问，Newproc 执行完毕后，父进程先返回用户态还是子进程先返回用户态？你确定嘛？

不确定父进程先返回用户态还是子进程先返回用户态！取决于 SetPri() 的结果

两种可能的情况

- 情况 A：父进程先返回

父进程执行 SetPri() 后，`priority <= CurPri`

RunRun 没有被设置

父进程直接返回用户态

时间线：

T0: 父进程调用 fork()，进入核心态

T1: NewProc() 复制子进程图像，返回 0

T2: Fork() 执行 else 分支，设置 EAX=子进程 PID

T3: Sys_Fork() 返回

T4: Trap() 执行 SetPri() → 没有设置 RunRun

T5: SystemCallEntrance() 检查 RunRun=0，不调用 Swtch()

T6: 父进程 iret 返回用户态

父进程在用户态继续运行

↓ (某个时刻，时钟中断等)

T7: 触发 Swtch()，Select() 选中子进程

T8: 子进程被调度上台，返回用户态

- 情况 B: 子进程先返回
父进程执行 SetPri()后, $\text{priority} > \text{CurPri}$
RunRun 被设置为非 0
SystemCallEntrance()中调用 Swtch()
Select()选中优先级更高的子进程 ($\text{p_pri}=0$)

时间线:

- T0: 父进程调用 fork(), 进入核心态
- T1: NewProc()复制子进程图像, 返回 0
- T2: Fork()执行 else 分支, 设置 EAX=子进程 PID
- T3: Sys_Fork()返回
- T4: Trap()执行 SetPri()
 - $\text{priority} = \text{p_cpu}/16 + \text{PUSER} + \text{p_nice}$
 - 如果 $\text{priority} > \text{CurPri}$, $\text{RunRun}++$
- T5: SystemCallEntrance()检查 $\text{RunRun} > 0$, 调用 Swtch()
- T6: Select()挑选进程:
 - 子进程 $\text{p_pri}=0$ (最高优先级)
 - 父进程 $\text{p_pri}=100+$ (可能更大)
 - 选中子进程
- T7: 切换到子进程, Swtch()返回 1
- T8: 子进程从 NewProc()的 SaveU 之后继续执行
- T9: NewProc()返回 1 → Fork()的 if 分支 → 设置 EAX=0
- T10: 子进程 iret 返回用户态
子进程在用户态运行...
↓ (某个时刻)
- T11: 父进程被调度上台
- T12: 父进程返回用户态